

Министерство образования Республики Беларусь

Учреждение образования «Белорусский государственный университет  
информатики и радиоэлектроники»

Факультет компьютерных систем и сетей

Кафедра электронных вычислительных машин

Дисциплина: Структурная и функциональная организация вычислительных  
машин

Отчет по лабораторной работе №4

«Считывание, декодирование и выполнение команд. Способ адресации  
операндов в командах»

Вариант 2

Выполнил: студент группы 250541  
Семененя К.Н.

Проверил: ассистент каф. ЭВМ,  
Воронов А.А.

Минск 2026

## Содержание

|                                                                                                |    |
|------------------------------------------------------------------------------------------------|----|
| 1 Задание .....                                                                                | 3  |
| 2 Проектирование требуемых блоков .....                                                        | 5  |
| 2.1 Проектирование блока регистра флагов .....                                                 | 5  |
| 2.1.1 Схема блока регистра флагов .....                                                        | 5  |
| 2.1.2 Моделирование блока регистра флагов .....                                                | 5  |
| 2.2 Проектирование блока регистра команд .....                                                 | 6  |
| 2.2.1 Схема блока регистра команд .....                                                        | 6  |
| 2.2.2 Моделирование блока регистра команд .....                                                | 7  |
| 2.3 Проектирование блока адресного регистра памяти .....                                       | 7  |
| 2.3.1 Схема блока адресного регистра памяти .....                                              | 7  |
| 2.3.2 Моделирование блока адресного регистра памяти .....                                      | 8  |
| 2.4 Проектирование блока счетчика команд со схемой вычисления<br>относительного перехода ..... | 9  |
| 2.4.1 Схема блока счетчика команд .....                                                        | 9  |
| 2.4.2 Моделирование блока счетчика команд .....                                                | 10 |
| 2.5 Проектирование блока буферного регистра данных .....                                       | 10 |
| 2.5.1 Схема блока буферного регистра данных .....                                              | 10 |
| 2.5.2 Моделирование блока буферного регистра данных .....                                      | 11 |
| 2.6 Проектирование блока регистров общего назначения .....                                     | 12 |
| 2.6.1 Схема блока регистров общего назначения .....                                            | 12 |
| 2.6.2 Моделирование блока регистров общего назначения .....                                    | 13 |
| 2.7 Проектирование блока памяти .....                                                          | 13 |
| 2.7.1 Схема блока памяти .....                                                                 | 13 |
| 2.7.2 Моделирование блока памяти .....                                                         | 14 |
| 2.8 Проектирование блока управления .....                                                      | 15 |
| 2.8.1 Схема блока управления .....                                                             | 15 |
| 2.8.2 Моделирование блока управления .....                                                     | 17 |
| 3 Проектирование общей схемы .....                                                             | 18 |
| 3.1 Схема взаимодействия спроектированных блоков .....                                         | 18 |
| 3.2 Моделирования общей схемы .....                                                            | 19 |

## 1 ЗАДАНИЕ

В процессе работы над лабораторной работой можно использовать наработки с предыдущих лабораторных работ.

На выходе должна получиться система, состоящая из блока памяти, блока управления, блока РОНов и шин данных, адреса и управления.

Для выполнения задания необходимо выполнить следующие этапы:

4.2.1. Ввести шину данных (ШД), шину адреса (ША) и шину управления(ШУ).

4.2.2. Разделить память на память данных (блок RAM) и память команд (блок ROM). На адресные входы памяти завести ША. Ввод и вывод данных осуществлять через ШД. Объединить два типа памяти в один блок.

4.2.3. Ввести блок управления, в котором производить дешифрацию команд и выдавать управляющие сигналы и сигналы синхронизации памяти и регистров в зависимости от выполняемой команды.

4.2.4. Ввести блок регистров общего назначения (РОН) и управляющую логику для него.

4.2.5. Ввести специальные регистры, разрядность которых

определяется разрядностью шины данных. Физически разместить их в блоке управления.

Специальные регистры:

1) Счетчик команды IP (Instruction Pointer), который служит для фиксации и формирования адресов команд, располагающихся в ячейках памяти. Необходимо обеспечить автоинкремент этого регистра.

2) Регистр команд IR (Instruction Register), который предназначен для хранения кода команды или его части в процессе выполнения команды после ее приема из памяти.

3) Регистр адреса операнда AR (Address Register), в который записывается исполнительный адрес операнда.

4) Регистр данных DR (Data Register), в который помещаются непосредственно сами операнды.

5) Регистр признаков результата или регистр флагов FR (Flag Register), в котором необходимо задать биты для переполнения, признака нуля, признака знака, переноса.

Интерфейс с внешней средой у верхнего блока иерархии должен ограничиваться старт/стоповым сигналом и сигналом внешнего тактирования clk.

Выполняемую команду можно разбить на несколько этапов. Для разделения этих этапов во времени можно использовать блок опорных сигналов DСa или блок генерации сигнала специальной формы, разработанные на лабораторной работе №2.

Необходимо выполнить цикл заданных по варианту команд. Цикл состоит из следующих фаз:

- выборки команды и формирования адреса следующей команды;
- декодирования команды;
- формирования исполнительных адресов операндов;
- выборки операндов;
- исполнения операции;
- записи результата.

Некоторые шаги цикла команд могут отсутствовать, это зависит от выполняемой команды.

Если также при анализе команды вы приходите к выводу, что вам не нужны некоторые специальные регистры в виду отсутствия действия для этого регистра, то в операции вы их не используете, но сама их реализация должна присутствовать.

Таблица 1.1 – Задание

| Разр. ША | Разр. ШД | Команда 1 | Адресация операндов команды 1 | Команда 2 | Адресация операндов команды 2 |
|----------|----------|-----------|-------------------------------|-----------|-------------------------------|
| 4        | 8        | M->R      | Прямая                        | JZ adr    | Относительная со смещением    |

## 2 ПРОЕКТИРОВАНИЕ ТРЕБУЕМЫХ БЛОКОВ

### 2.1 Проектирование блока регистра флагов

#### 2.1.1 Схема блока регистра флагов

Структурная схема блока регистра флагов представлена на рисунке 2.1.

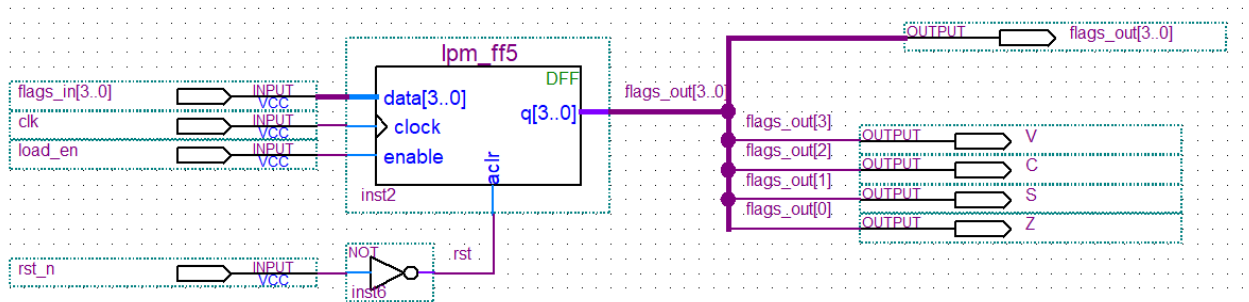


Рисунок 2.1 – Структурная схема блока регистра флагов

Блок основан на параллельном 4-разрядном D-регистре lpm\_ff5, синхронизированном с входным тактовым сигналом clk. Запись входного 4-битного вектора признаков flags\_in[3..0] осуществляется по фронту clk при условии наличия разрешающего сигнала на входе load\_en. Цепь асинхронного сброса, реализованная через инвертор NOT, преобразует внешний сигнал rst\_n для входа asclr, принудительно обнуляя регистр при старте системы. На выходе блока сформирована шина flags\_out[3..0], которая разделяется на четыре независимых статус-флага: переполнения (V), переноса (C), знака (S) и нуля (Z). В результате блок обеспечивает стабильное хранение и выдачу признаков результата операции для автомата управления, где флаг Z используется как логическое условие для выполнения команды условного перехода JZ adr.

#### 2.1.2 Моделирование блока регистра флагов

Результаты функционального моделирования блока регистра флагов приведены на рисунке 2.2.

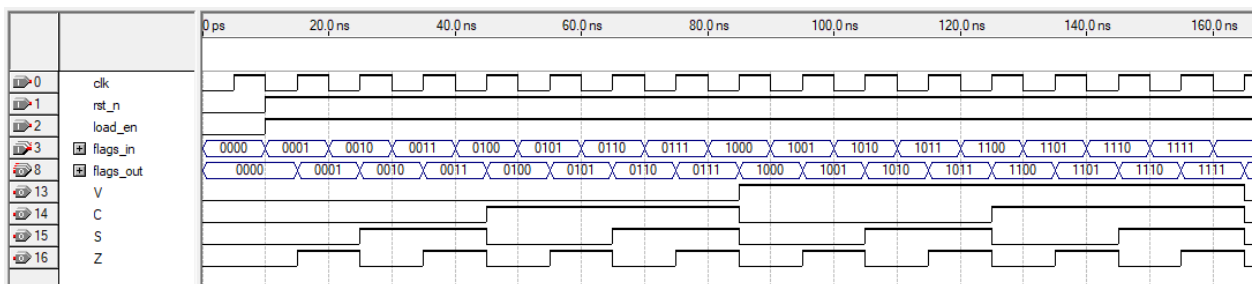


Рисунок 2.2 – Результаты моделирования блока регистра флагов

Перед началом симуляции по сигналу  $\text{rst\_n} = 0$  производится асинхронный сброс, переводящий шину  $\text{flags\_out}$  и отдельные выходы V, C, S, Z в исходное нулевое состояние. При подаче активного уровня разрешения записи  $\text{load\_en} = 1$  блок начинает синхронно фиксировать изменения входного вектора  $\text{flags\_in}[3..0]$ . На временной диаграмме видно, что при последовательном переборе входных комбинаций от 0000 до 1111 данные перезаписываются в регистр строго по каждому положительному фронту тактового сигнала  $\text{clk}$ . Выходная шина  $\text{flags\_out}$  стабильно дублирует входные значения с задержкой в один такт. При этом логические уровни на индивидуальных портах флагов полностью соответствуют двоичному коду вектора: младший бит определяет поведение флага нуля (Z), второй — знака (S), третий — переноса (C), а старший — переполнения (V).

## 2.2 Проектирование блока регистра команд

### 2.2.1 Схема блока регистра команд

Структурная схема блока регистра флагов представлена на рисунке 2.3.

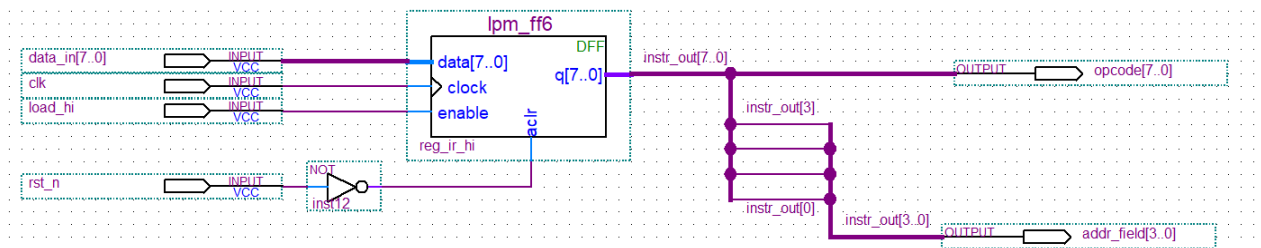


Рисунок 2.3 – Структурная схема блока регистра команд

Блок основан на параллельном 8-разрядном D-регистре  $\text{lpm\_ff6}$  ( $\text{reg\_ir\_hi}$ ), синхронизированном с входным тактовым сигналом  $\text{clk}$ . Запись 8-битного кода считанной из памяти команды с шины  $\text{data\_in}[7..0]$  осуществляется по фронту  $\text{clk}$  при условии наличия разрешающего сигнала на входе  $\text{load\_hi}$ . Цепь асинхронного сброса, реализованная через инвертор NOT ( $\text{inst12}$ ), преобразует внешний сигнал  $\text{rst\_n}$  для входа  $\text{asclr}$ , принудительно обнуляя регистр при старте системы. Выходная 8-битная шина  $\text{instr\_out}[7..0]$  разделяется на два независимых информационных потока: старшие разряды напрямую формируют полный код операции  $\text{opcode}[7..0]$  для дешифратора, а младшие разряды  $\text{instr\_out}[3..0]$  выделяются в отдельную шину  $\text{addr\_field}[3..0]$ . В результате блок обеспечивает стабильное хранение текущей инструкции и разделение ее на управляющую часть (код операции) и адресное поле операнда (смещение для относительного перехода JZ  $\text{adr}$  или прямой адрес для команды M->R).

## 2.2.2 Моделирование блока регистра команд

Результаты функционального моделирования блока регистра команд приведены на рисунке 2.4.

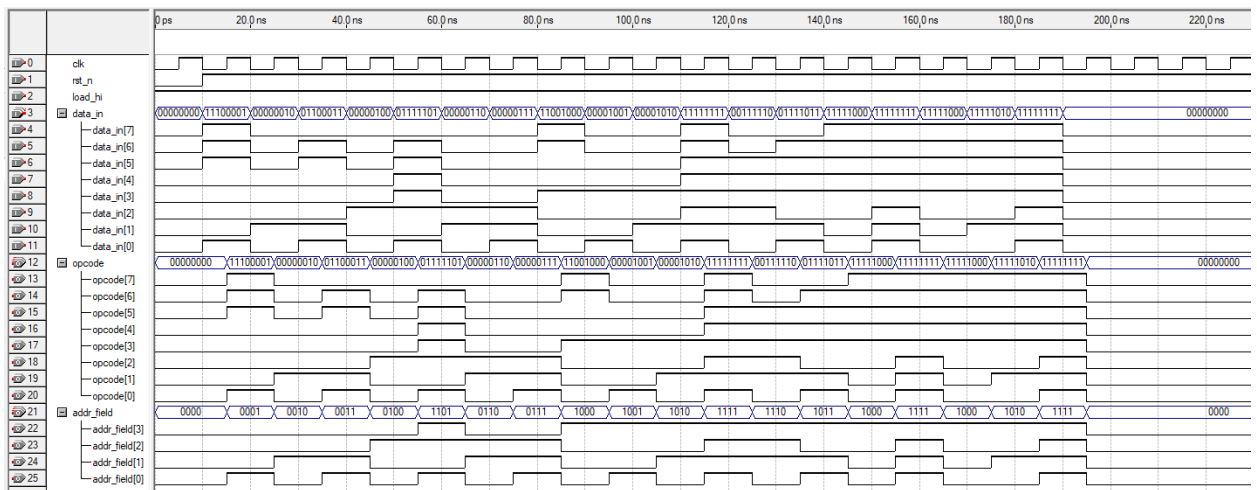


Рисунок 2.4 – Результаты моделирования блока регистра команд

Перед началом симуляции по сигналу  $rst_n = 0$  производится асинхронный сброс, переводящий выходной код операции `opcode` и адресное поле `addr_field` в исходное нулевое состояние. При подаче активного уровня разрешения записи `load_hi = 1` блок начинает синхронно фиксировать изменения 8-битного входного вектора `data_in[7..0]`. На временной диаграмме видно, что при последовательной подаче кодов команд на шину ввода, данные перезаписываются в регистр строго по каждому положительному фронту тактового сигнала `clk`. Выходная шина `opcode[7..0]` полностью дублирует записанные значения с задержкой в один такт. При этом младшие разряды вектора успешно выделяются в отдельный поток `addr_field[3..0]`, логические уровни которого точно соответствуют младшей тетраде входных данных (например, при вводе команды 11100001 поле адреса принимает значение 0001, а при 11000010 — 0010).

## 2.3 Проектирование блока адресного регистра памяти

### 2.3.1 Схема блока адресного регистра памяти

Структурная схема блока адресного регистра памяти представлена на рисунке 2.5.

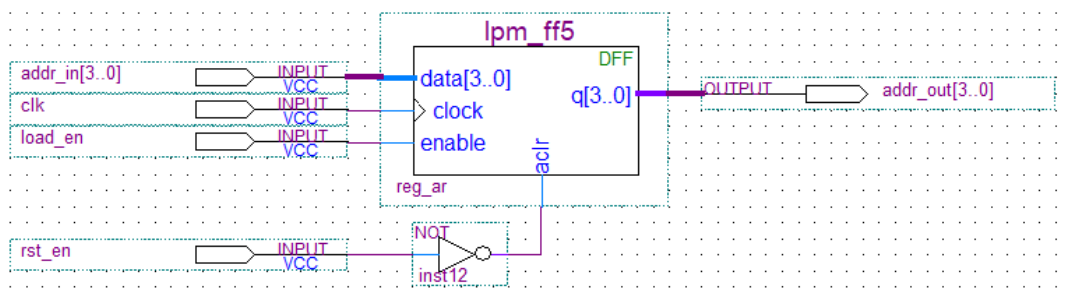


Рисунок 2.5 – Структурная схема блока адресного регистра памяти

Блок основан на параллельном 4-разрядном D-регистре lpm\_ff5 (reg\_ar), синхронизированном с входным тактовым сигналом clk. Запись 4-битного исполнительного адреса памяти с шины addr\_in[3..0] осуществляется по фронту clk при условии наличия разрешающего сигнала на входе load\_en. Цепь асинхронного сброса, реализованная через инвертор NOT (inst12), преобразует внешний сигнал rst\_n для входа asclr, принудительно обнуляя регистр при старте системы. Выходная 4-битная шина addr\_out[3..0] напрямую подключена к выходному порту устройства, обеспечивая стабильное удержание адреса ячейки ОЗУ или ПЗУ во время фаз чтения или записи данных.

### 2.3.2 Моделирование блока адресного регистра памяти

Результаты функционального моделирования блока адресного регистра памяти приведены на рисунке 2.6.

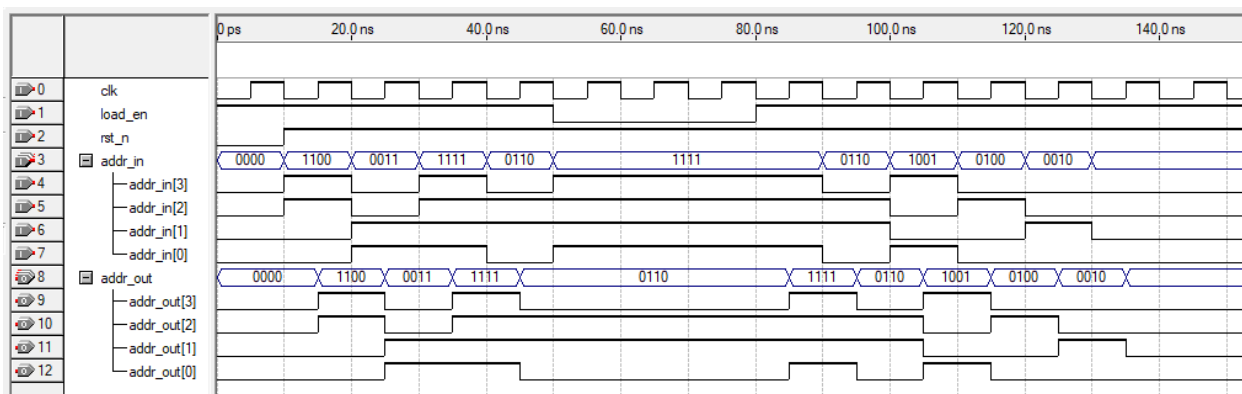


Рисунок 2.6 – Результаты моделирования блока адресного регистра памяти

Перед началом симуляции по сигналу  $\text{rst}_n = 0$  производится асинхронный сброс, переводящий выходную шину  $\text{addr\_out}[3..0]$  в исходное нулевое состояние. При подаче активного уровня разрешения записи  $\text{load\_en} = 1$  блок начинает синхронно фиксировать изменения 4-битного входного вектора  $\text{addr\_in}[3..0]$ , обновляя данные строго по каждому положительному



фронту тактового сигнала  $clk$ . На временной диаграмме наглядно продемонстрирован режим блокировки памяти: при сбросе сигнала  $load\_en$  в логический «0» на интервале от 55 до 90 нс, регистр прекращает реагировать на изменение входной шины  $addr\_in$  (переход значения с 0110 на 1111) и стабильно удерживает на выходе ранее сохраненный адрес 0110. После восстановления высокого уровня  $load\_en = 1$  возобновляется штатная синхронная перезапись входных векторов (1111, 0110 и т.д.) с задержкой в один такт.

## 2.4 Проектирование блока счетчика команд со схемой вычисления относительного перехода

### 2.4.1 Схема блока счетчика команд

Структурная схема блока счетчика команд представлена на рисунке 2.7.

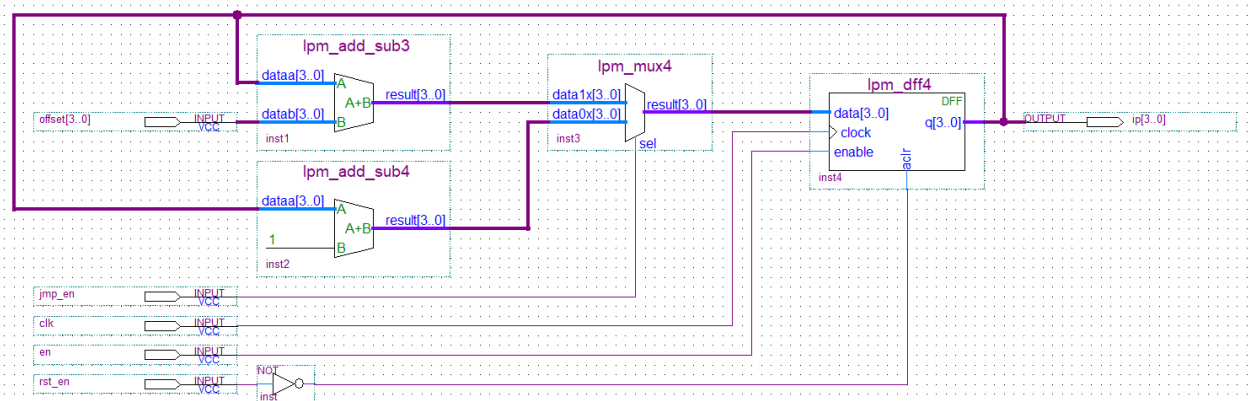


Рисунок 2.7 – Структурная схема блока счетчика команд

Блок основан на параллельном 4-разрядном D-регистре  $lpm\_dff4$  (inst4), синхронизированном с входным тактовым сигналом  $clk$ . Запись вычисленного адреса команды осуществляется по фронту  $clk$  при наличии разрешающего сигнала на входе  $enable$ . Цепь асинхронного сброса через инвертор NOT (inst5) преобразует внешний сигнал  $rst\_n$  для входа  $aclr$ , обнуляя регистр при старте системы. Текущий адрес с выхода  $q[3..0]$  возвращается по цепи обратной связи на комбинационную логику, состоящую из двух сумматоров и мультиплексора  $lpm\_mux4$  (inst3), управляемого сигналом  $jmp\_en$ .

В линейном режиме работы ( $jmp\_en = 0$ ) мультиплексор выбирает выход сумматора инкремента  $lpm\_add\_sub4$  (inst2), который постоянно прибавляет единицу к текущему адресу. В режиме условного перехода ( $jmp\_en = 1$  для вашей команды JZ adr) мультиплексор переключается на выход сумматора смещения  $lpm\_add\_sub3$  (inst1), который складывает текущий адрес с 4-битным значением относительного смещения  $offset[3..0]$ .

## 2.4.2 Моделирование блока счетчика команд

Результаты функционального моделирования блока счетчика команд приведены на рисунке 2.8.

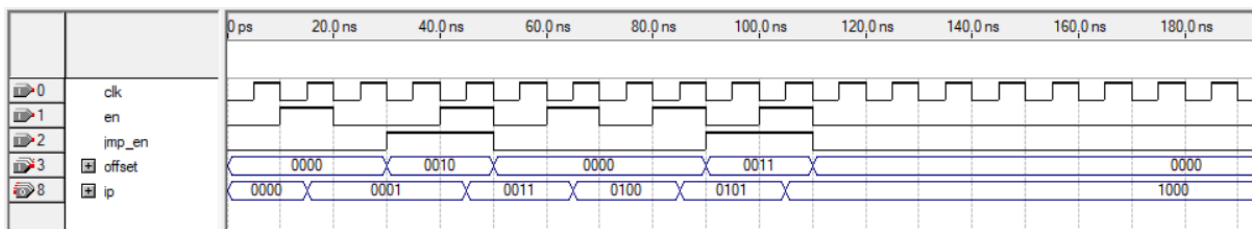


Рисунок 2.8 – Результаты моделирования блока счетчика команд

Перед началом симуляции производится асинхронный сброс, устанавливающий выходной адрес ip в исходное нулевое состояние 0000. При подаче сигнала разрешения работы en = 1 в линейном режиме (jmp\_en = 0) блок осуществляет последовательный инкремент адреса строго по положительным фронтам тактового сигнала clk (переход 0000 -> 0001).

На временной диаграмме наглядно продемонстрирован режим относительного перехода: при подаче управляющего сигнала jmp\_en = 1 на интервале от 35 до 60 нс мультиплексор переключает логику вычисления на сумматор смещения, в результате чего по очередному такту clk к текущему адресу 0001 прибавляется вектор offset = 0010, и значение ip скачкообразно принимает значение 0011. При снятии сигнала jmp\_en на интервале от 60 до 90 нс счетчик возвращается к линейному инкременту (0011 -> 0100 -> 0101). Повторное включение jmp\_en = 1 на интервале от 90 до 115 нс со смещением offset = 0011 успешно переводит счетчик в адрес 1000.

## 2.5 Проектирование блока буферного регистра данных

### 2.5.1 Схема блока буферного регистра данных

Структурная схема блока буферного регистра данных представлена на рисунке 2.9.

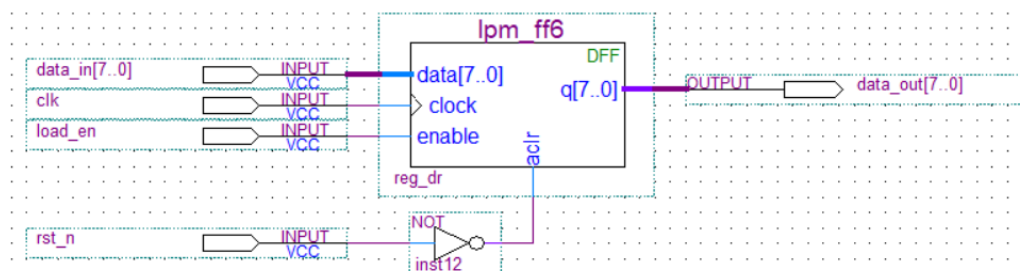


Рисунок 2.9 – Структурная схема блока буферного регистра данных

Блок основан на параллельном 8-разрядном D-регистре `lpm_ff6` (`reg_dr`), синхронизированном с входным тактовым сигналом `clk`. Запись 8-битного операнда данных со сквозной шины памяти `data_in[7..0]` осуществляется по фронту `clk` при условии наличия разрешающего сигнала на входе `load_en`. Цепь асинхронного сброса, реализованная через инвертор NOT (`inst12`), преобразует внешний сигнал `rst_n` для входа `aclr`, принудительно обнуляя регистр при старте системы. Выходная 8-битная шина `data_out[7..0]` напрямую подключена к выходному порту устройства, обеспечивая стабильное удержание считанного из ОЗУ числа.

## 2.5.2 Моделирование блока буферного регистра данных

Результаты функционального моделирования блока буферного регистра данных приведены на рисунке 2.10.

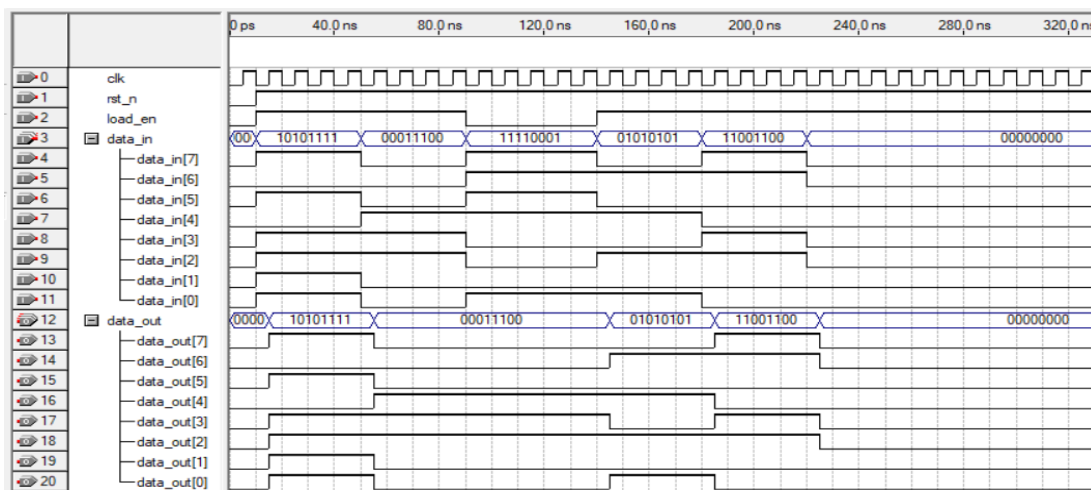


Рисунок 2.10 – Результаты моделирования блока буферного регистра данных

Перед началом симуляции по сигналу `rst_n = 0` производится асинхронный сброс, переводящий выходную шину `data_out[7..0]` в исходное нулевое состояние. При подаче активного уровня разрешения записи `load_en = 1` блок начинает синхронно фиксировать изменения 8-битного входного вектора `data_in[7..0]`, обновляя выходные данные строго по каждому положительному фронту тактового сигнала `clk` (запись векторов 10101111 и 00011100).

На временной диаграмме наглядно продемонстрирован режим изоляции шины: при сбросе сигнала `load_en` в логический «0» на интервале от 105 до 165 нс, регистр прекращает реагировать на появление нового входного слова 11110001 и стабильно удерживает на выходе ранее сохраненное значение 00011100. После восстановления высокого уровня `load_en = 1` на отметке 165 нс возобновляется штатная синхронная перезапись входных векторов с задержкой в один такт (01010101, 11001100 и т.д.).

## 2.6 Проектирование блока регистров общего назначения

### 2.6.1 Схема блока регистров общего назначения

Структурная схема блока регистров общего назначения представлена на рисунке 2.11.

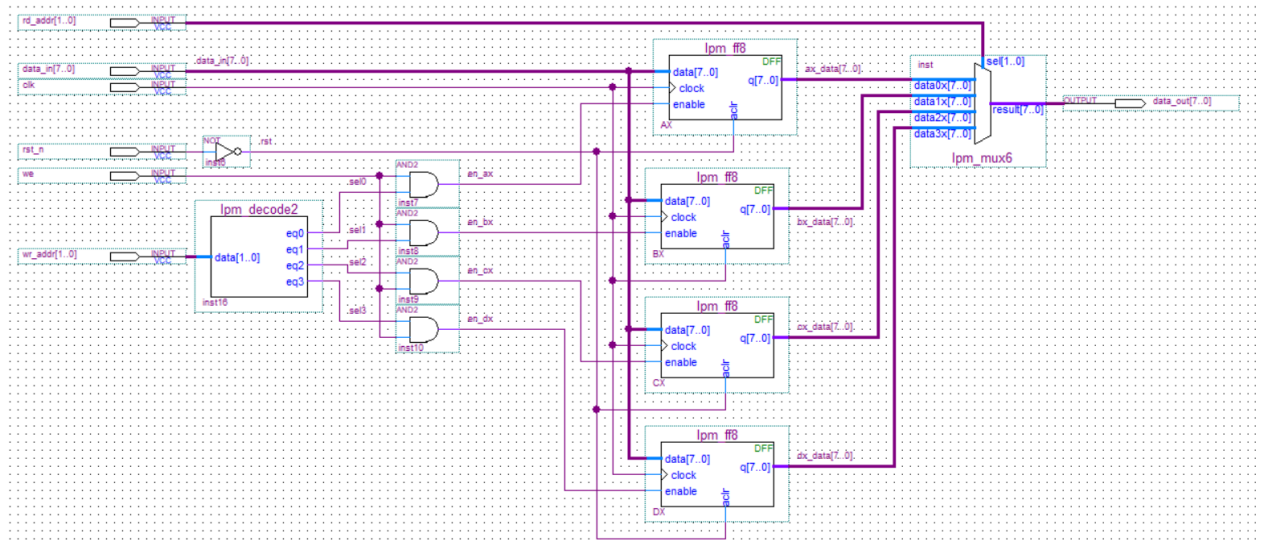


Рисунок 2.11 – Структурная схема блока регистров общего назначения

Блок представляет собой упорядоченный массив регистров общего назначения (AX, BX, CX, DX), реализованный на четырех параллельных 8-разрядных D-регистрах `lpm_ff8`, синхронизированных с входным тактовым сигналом `clk`. Цепь асинхронного сброса, выполненная через инвертор NOT, объединяет входы `aslr` всех четырех модулей памяти, обеспечивая их одновременное и полное обнуление при активации сигнала `rst_n`. Назначение целевого регистра для операции синхронной записи операнда с шины `data_in[7..0]` осуществляется с помощью 2-разрядного дешифратора адреса `lpm_decode2` (`wr_addr[1..0]`), выходы которого объединены через логические элементы AND2 с общим сигналом разрешения записи `we`. В результате стробирующий импульс записи (`en_ax–en_dx`) поступает строго на вход `enable` выбранного регистра.

Для организации асинхронного считывания данных используется 8-разрядный мультиплексор 4 в 1 `lpm_mux6`, коммутирующий выходные шины регистров (`ax_data–dx_data`) на общий выходной порт `data_out[7..0]` в соответствии с кодом на адресной шине чтения `rd_addr[1..0]`. Модифицированная топология блока обеспечивает полностью идентичное и независимое функционирование всех регистров РОН, что гарантирует бесконфликтную прямую адресацию данных при выполнении команды `M->R`.

## 2.6.2 Моделирование блока регистров общего назначения

Результаты функционального моделирования блока регистров общего назначения (РОН) приведены на рисунке 2.12.

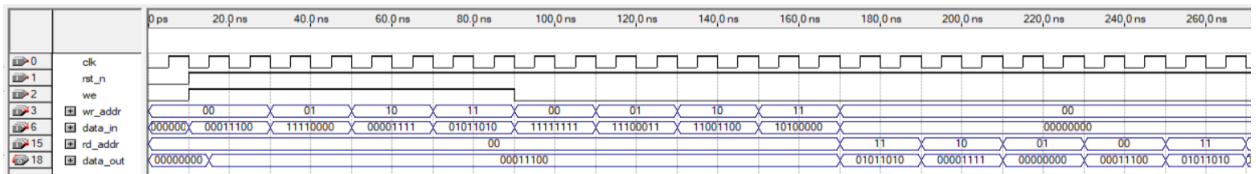


Рисунок 2.12 – Результаты моделирования блока РОН

Перед началом симуляции по сигналу `rst_n = 0` производится полный асинхронный сброс, одновременно обнуляющий внутреннее содержимое всех четырех регистров общего назначения. При активации управляющего сигнала разрешения записи `we = 1` на интервале от 10 до 90 нс блок осуществляет последовательную фиксацию входных 8-битных операндов по фронту тактового сигнала `clk` в соответствии с адресом `wr_addr[1..0]`. На временной диаграмме видно, что данные безошибочно защелкиваются в свои целевые регистры.

Начиная с 90 нс, сигнал `we` сбрасывается в «0», блокируя дальнейшую перезапись, а на шине `rd_addr[1..0]` иницируется цикл асинхронного чтения. Выходной мультиплексор мгновенно коммутирует сохраненное содержимое выбранных регистров на общую шину `data_out[7..0]` без задержки на тактовые импульсы: при адресе 11 на выход выдаются данные из DX, при 10 – из CX, при 01 – из VX, а при 00 – из AX.

## 2.7 Проектирование блока памяти

### 2.7.1 Схема блока памяти

Структурная схема блока памяти представлена на рисунке 2.13.

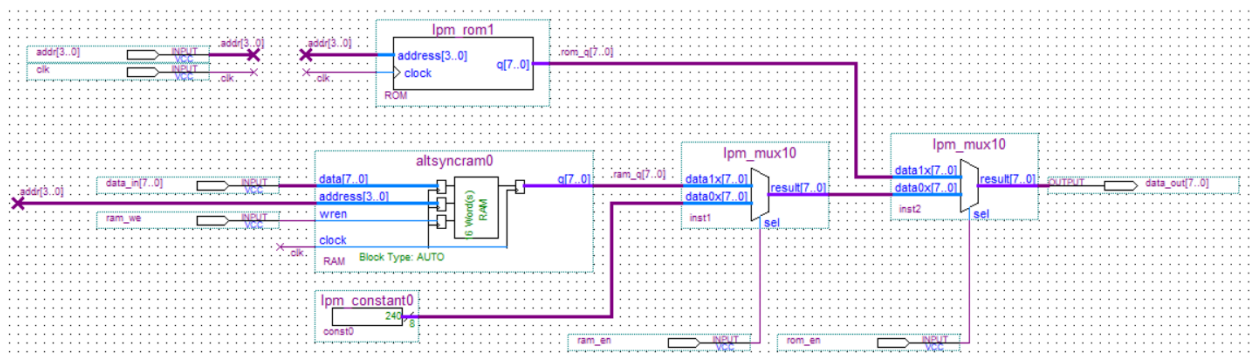


Рисунок 2.13 – Структурная схема блока памяти

Блок основан на совмещении модулей постоянного хранения данных lpm\_rom1 (ПЗУ команд) и оперативной памяти altsyncram0 (ОЗУ данных), синхронизированных с входным тактовым сигналом clk. Модуль ПЗУ осуществляет чтение кодов инструкций по 4-разрядной шине адреса addr[3..0]. Модуль ОЗУ обеспечивает синхронную запись 8-битных операндов со сквозной шины data\_in[7..0] при подаче сигнала разрешения записи ram\_we на вход wren. Схема также содержит блок аппаратной константы lpm\_constant0, генерирующий постоянное 8-разрядное значение 240.

Объединение выходов всех трех источников в общую 8-разрядную выходную шину данных data\_out[7..0] реализовано на каскаде двух мультиплексоров lpm\_mux10 (inst1 и inst2). В зависимости от фазы выполнения команды, автомат управления выставляет сигналы выбора ram\_en и rom\_en. При rom\_en = 1 мультиплексоры коммутируют на выход код команды из ПЗУ, при ram\_en = 1 — данные из ОЗУ для команды M->R, а при пассивных уровнях ram\_en = 0 и rom\_en = 0 на шину выставляется значение константы. В результате блок обеспечивает бесконфликтное разделение адресного пространства и управление потоками команд и операндов на единой системной шине данных.

## 2.7.2 Моделирование блока памяти

Результаты функционального моделирования блока памяти приведены на рисунках 2.14 – 2.16.

| Addr | +0 | +1 | +2 | +3 | +4 | +5 | +6 | +7 |
|------|----|----|----|----|----|----|----|----|
| 0    | 12 | 23 | 00 | 00 | 11 | 00 | 00 | 00 |
| 8    | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 |

Рисунок 2.14 – Хранящиеся данные в ПЗУ

| Addr | +0 | +1 | +2 | +3 | +4 | +5 | +6 | +7 |
|------|----|----|----|----|----|----|----|----|
| 0    | XX | 1A | 06 | XX | XX | XX | XX | XX |
| 8    | XX | XX | XX | XX | XX | XX | XX | XX |

Рисунок 2.15 – Записанные данные в ОЗУ

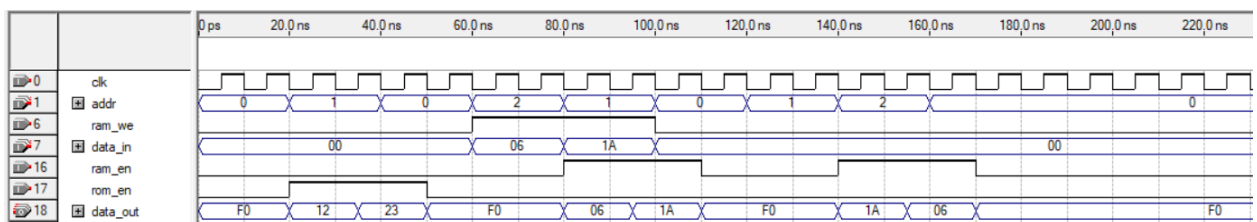


Рисунок 2.16 – Результаты моделирования памяти



При пассивных уровнях управляющих сигналов  $rom\_en=0$  и  $ram\_en=0$  каскад мультиплексоров успешно коммутирует на выходной порт аппаратную константу блока  $lpm\_constant0$ , фиксируя на шине значение F0.

На временной диаграмме продемонстрированы три основных режима работы подсистемы памяти.

Режим чтения из ПЗУ. При активации сигнала  $rom\_en = 1$  блок осуществляет выборку инструкций по шине  $addr$  строго по фронтам тактового сигнала  $clk$ . По адресу 0 считывается код команды 12, а по адресу 1 — код перехода 23.

Режим синхронной записи в ОЗУ. При поднятии сигнала разрешения записи  $ram\_we = 1$  производится фиксация входных операндов с шины  $data\_in$ : число 06 успешно записывается в ячейку ОЗУ по адресу 2, а число 1A — по адресу 1. На шине  $data\_out$  в моменты записи кратковременно дублируются сохраняемые данные.

Режим асинхронного чтения из ОЗУ. При снятии сигнала записи ( $ram\_we = 0$ ) и удержании  $ram\_en = 1$  мультиплексоры переключаются на чтение из ОЗУ. На выходе  $data\_out$  безошибочно появляются ранее сохраненные операнды в соответствии с текущим адресом на шине  $addr$ : значение 1A для адреса 1 и значение 06 для адреса 2.

## 2.8 Проектирование блока управления

### 2.8.1 Схема блока управления

Для разделения операций во времени использован блок опорных сигналов DCa, разработанные на лабораторной работе №2 (рисунок 2.17).

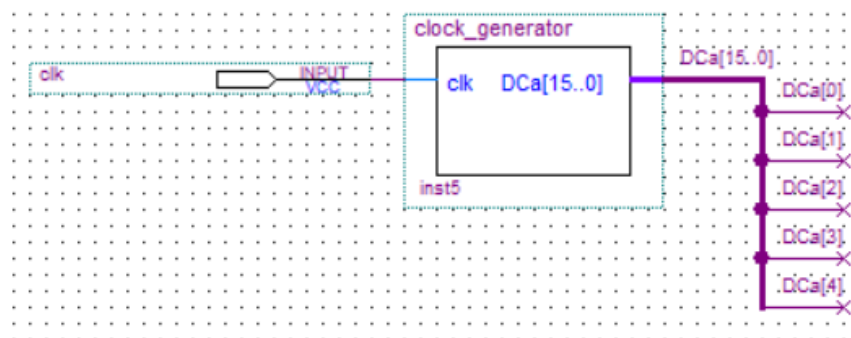


Рисунок 2.17 – Схема блока опорных сигналов

Структурная схема блока управления представлена на рисунке 2.18.

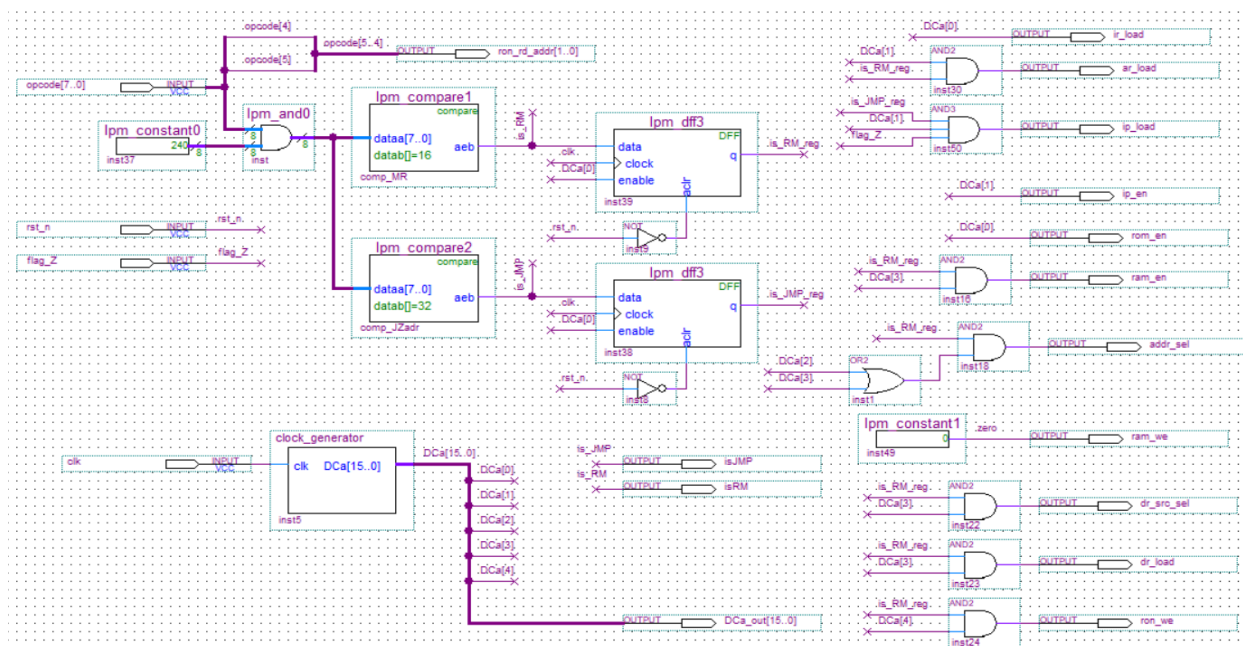


Рисунок 2.18 – Схема блока управления

Блок основан на дешифраторе команд и циклическом распределителе тактовых импульсов clock\_generator (inst5), синхронизированных с входным тактовым сигналом clk. Модуль распределителя поочередно генерирует сигналы фаз выполнения команды. На вход дешифратора поступает 8-разрядный код операции opcode[7..0]. С помощью логического элемента lpm\_and0 (inst) и маскирующей константы 240 отсекаются младшие разряды инструкции, а выделенный код операции подается на цифровые компараторы lpm\_compare1 и lpm\_compare2. Компаратор comp\_MR идентифицирует код 16 и формирует признак команды чтения is\_RM, а компаратор comp\_JZadr идентифицирует код 32 и формирует признак условного перехода is\_JMP. Полученные признаки синхронно фиксируются триггерами lpm\_dff3 на фазе выборки DCa0 и удерживаются в виде сигналов is\_RM\_reg и is\_JMP\_reg.

Формирование выходных управляющих сигналов для исполнительных узлов процессора реализовано на комбинационной логике элементов AND, OR и NOT, связывающих текущую фазу цикла с типом команды. При выполнении команды M->R схема последовательно генерирует импульсы загрузки адреса ar\_load, включения ОЗУ ram\_en, коммутации адреса addr\_sel и фиксации данных dr\_load, а также финальный сигнал записи в регистровый файл ron\_we. При выполнении команды условного перехода JZ adr комбинационная логика анализирует состояние входного признака нуля flag\_Z: в случае flag\_Z = 1 на фазе DCa1 элемент AND3 (inst3) вырабатывает сигнал принудительной загрузки адреса перехода ip\_load, а при flag\_Z = 0 формируется стандартный синхросигнал линейного шага ip\_en на фазе DCa0. В результате блок обеспечивает жесткую временную координацию межрегистровых передач и корректное ветвление программы в соответствии со спецификацией команд процессора.



## 2.8.2 Моделирование блока управления

Результаты функционального моделирования блока управления приведены на рисунке 2.19.

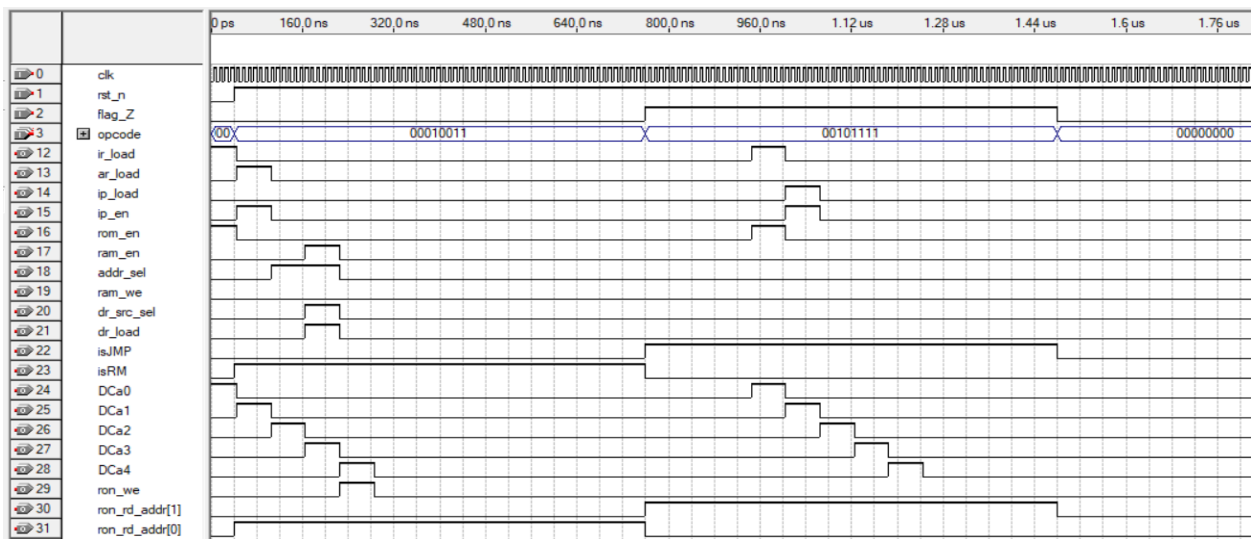


Рисунок 2.19 – Результаты моделирования блока управления

В качестве опорных сигналов используется циклический распределитель импульсов, поочередно генерирующий активные уровни фаз от DCa0 до DCa4 длительностью в один такт clk каждая.

На временной диаграмме наглядно продемонстрированы два полных цикла управления процессором.

На вход поступает код операции 00010011. Старшая тетрада дешифрируется компаратором, активируя внутренний признак isRM = 1. Логический автомат синхронно распределяет сигналы по фазам: на этапе DCa0 формируется строб выборки команды ir\_load и инкремент счетчика ip\_en. На фазе DCa1 генерируется импульс фиксации адреса операнда ar\_load. На фазе DCa2 комбинационная логика одновременно поднимает сигналы работы с ОЗУ ram\_en, переключения шины адреса addr\_sel, выбора источника dr\_src\_sel и защелкивания буфера данных dr\_load. Цикл завершается на фазе DCa4 выдачей строба записи в регистровый файл ron\_we при неизменном пассивном уровне ram\_we = 0 (режим чтения).

На шину opcode подается код инструкции перехода 00101111, переводящий автомат в режим isJMP = 1. На интервале симуляции входной флаг нуля находится в состоянии flag\_Z = 1, что означает выполнение логического условия ветвления. При совпадении фазы DCa1 и активного признака нуля комбинационная логика блокирует стандартные цепи адресации и генерирует единичный синхроимпульс принудительной загрузки адреса перехода ip\_load = 1. На последующих этапах DCa2–DCa4 все исполнительные сигналы памяти и регистров РОН (ram\_en, dr\_load, ron\_we) удерживаются в пассивном нулевом состоянии.

## 3 ПРОЕКТИРОВАНИЕ ОБЩЕЙ СХЕМЫ

### 3.1 Схема взаимодействия спроектированных блоков

Структурная схема взаимодействия спроектированных блоков представлена на рисунке 3.1.

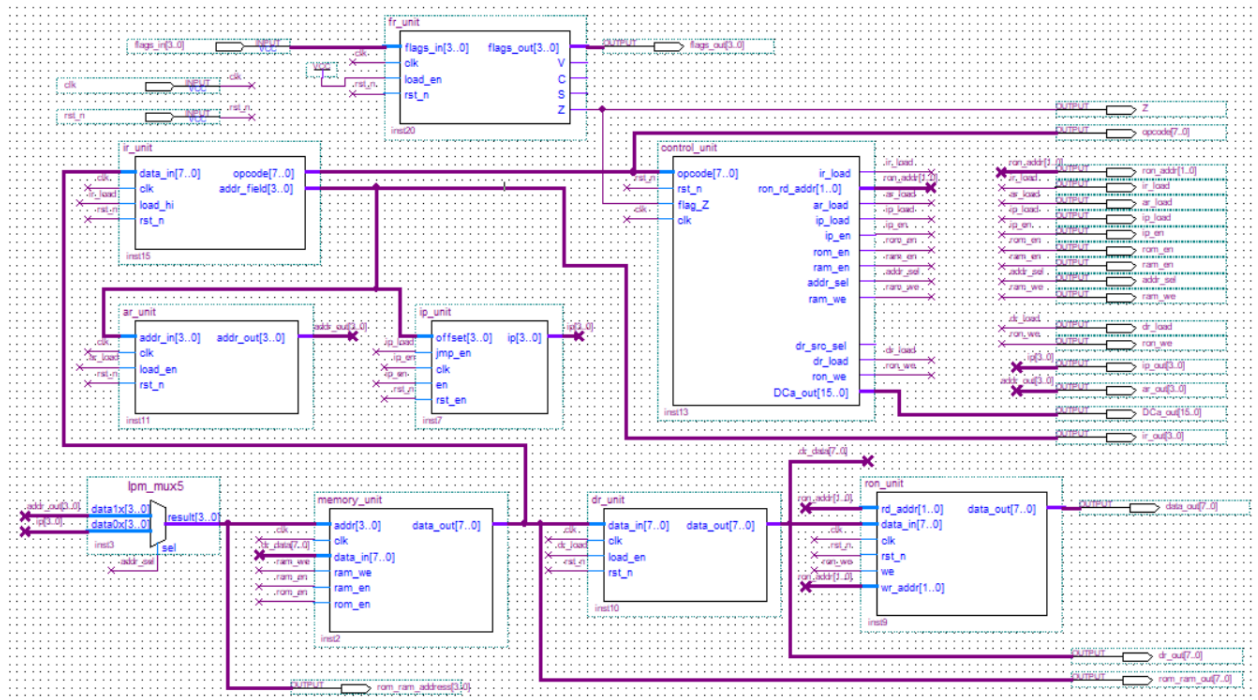


Рисунок 3.1 – Структурная схема взаимодействия спроектированных блоков

Данная схема объединяет все ранее спроектированные функциональные узлы в единый конвейер обработки команд. Сборка выполнена по модульному принципу и включает в себя следующие основные блоки: блок регистра флагов (fr\_unit), регистр команд (ir\_unit), адресный регистр памяти (ar\_unit), счетчик команд со схемой вычисления перехода (ip\_unit), объединенный блок памяти ОЗУ/ПЗУ (memory\_unit), буферный регистр данных (dr\_unit), регистровый файл (ron\_unit) и центральный блок управления (control\_unit). Взаимодействие между модулями организовано посредством развитой системы внутренних информационных шин данных, адресов и синхронизирующих сигналов управления.

Схема обеспечивает сквозную логику функционирования процессора для двух команд:

реализация команды M->R (Прямая адресация);

Реализация команды JZ adr (Относительный переход со смещением).

Спроектированная топология межмодульных связей обеспечивает полную бесконфликтность совместного использования системных шин, гарантирует жесткую временную синхронизацию информационных потоков от общего генератора тактовых импульсов clk.

## 3.2 Моделирования общей схемы

Результаты функционального моделирования блока памяти приведены на рисунках 3.2 – 3.5.

| Addr | +0 | +1 | +2 | +3 | +4 | +5 | +6 | +7 |
|------|----|----|----|----|----|----|----|----|
| 0    | 12 | 23 | 00 | 00 | 11 | 00 | 00 | 00 |
| 8    | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 |

Рисунок 3.2 – Хранящиеся данные в ПЗУ

| Addr | +0 | +1 | +2 | +3 | +4 | +5 | +6 | +7 |
|------|----|----|----|----|----|----|----|----|
| 0    | 07 | 05 | 08 | 09 | 05 | 06 | 07 | 00 |
| 8    | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 |

Рисунок 3.3 – Хранящиеся данные в ОЗУ

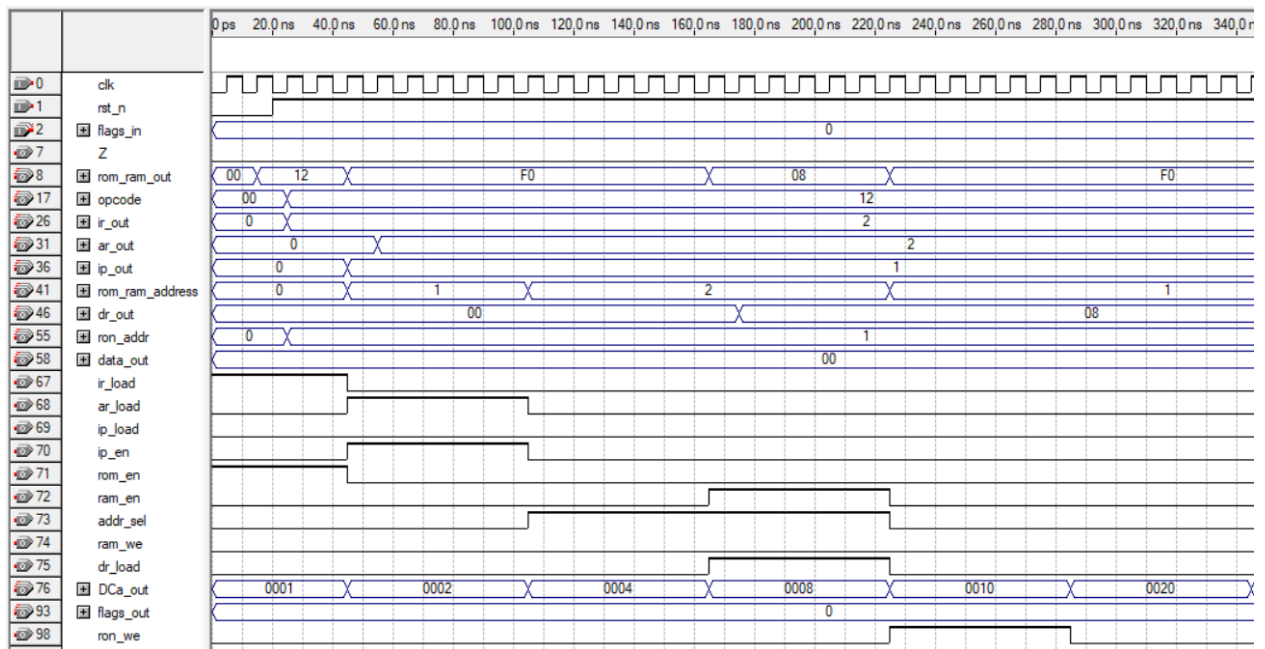


Рисунок 3.4 – Результаты выполнения команды M->R (Прямая адресация)

Временная диаграмма на рисунке 3.4 отражает полный пятитактный цикл работы управляющего автомата, координирующий последовательное извлечение, декодирование инструкции и пересылку данных из ОЗУ в регистровый файл.

На фазе выборки команды ( $DCa\_out = 0001$ ) блок управления активирует сигналы  $rom\_en = 1$  и  $ir\_load = 1$ . Со сквозной шины памяти  $rom\_ram\_out$  считывается код команды 12, транслируемый из ПЗУ по текущему адресу счетчика  $ip\_out = 0$ . По срезу импульса  $ir\_load$  код операции защелкивается в регистре команд ( $opcode$  принимает значение 12), а адрес

целевого регистра РОН коммутируется на шину  $ron\_addr = 1$ . Сигнал  $ip\_en = 1$  осуществляет инкремент счетчика команд ( $ip\_out$  переходит с 0 на 1).

На фазе фиксации адреса операнда ( $DCa\_out = 0002$ ) автомат генерирует импульс  $ar\_load = 1$ . Младшая тетрада считанной инструкции (число 2) из адресного поля регистра команд переписывается во внутренний адресный регистр памяти ( $ar\_out$  становится равным 2). Память команд в этот момент отключается, и на шине  $rom\_ram\_out$  временно выставляется код аппаратной константы 240.

На фазе чтения данных из ОЗУ ( $DCa\_out = 0004$  и  $0008$ ) управляющий блок формирует сигналы  $ram\_en = 1$  и  $addr\_sel = 1$ . Мультиплексор адреса памяти отключает счетчик команд  $ip\_unit$  и подает на шину  $rom\_ram\_address$  исполнительный адрес 2 из регистра  $ar\_out$ . Модуль оперативной памяти выдает на системную шину  $rom\_ram\_out$  хранящийся по этому адресу операнд 08. По сигналу  $dr\_load = 1$  на отметке 180 нс это число успешно фиксируется буферным регистром данных ( $dr\_out$  принимает значение 08).

На фазе записи в РОН ( $DCa\_out = 0010$ ) на финальном этапе исполнительного цикла автомат выдает стробирующий импульс записи  $ron\_we = 1$ . Число 08 из буферного регистра  $dr\_out$  успешно переписывается в регистровый файл по ранее декодированному адресу  $ron\_addr = 1$  (запись в регистр ВХ), о чем свидетельствует появление значения 08 на общей выходной шине данных  $data\_out$ . Последующий такт  $DCa\_out = 0020$  завершает выполнение текущей команды и переводит процессор в состояние ожидания нового цикла.

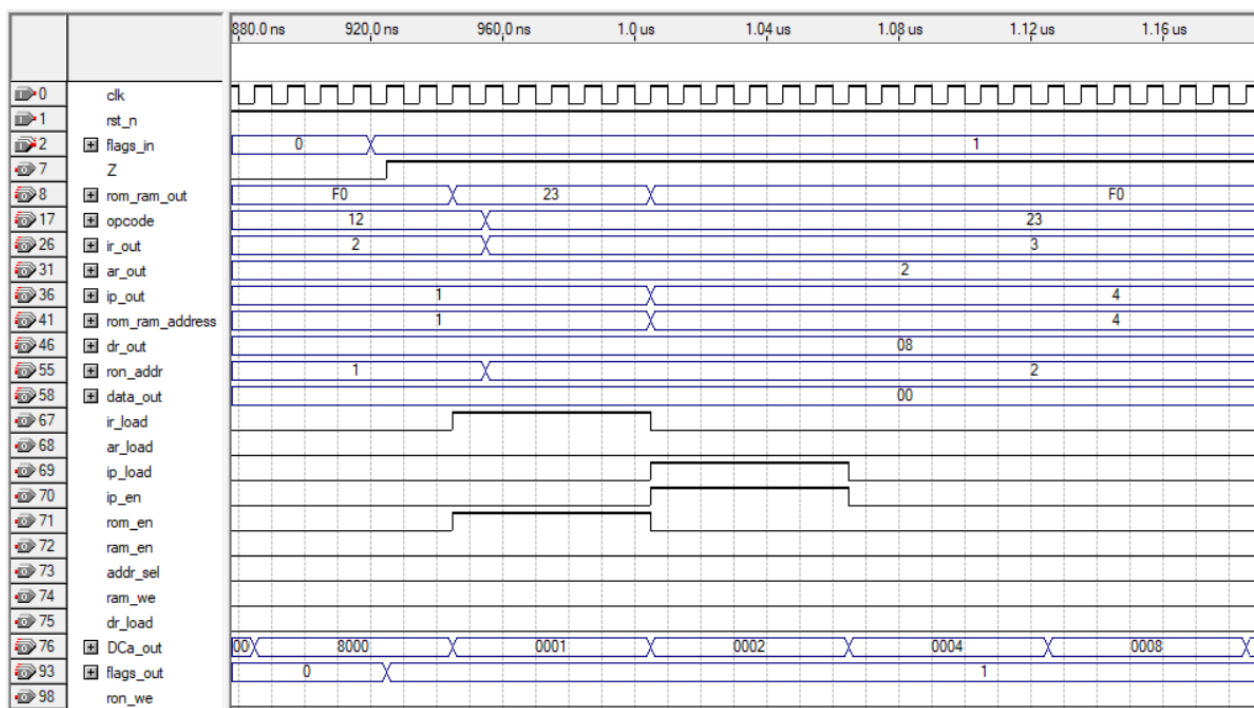


Рисунок 3.5 – Результаты выполнения команды JZ adr (Относительный переход со смещением)

Моделирование на рисунке 3.5 подтверждает правильность взаимодействия регистра флагов `fr_unit`, счетчика команд `ip_unit` и центрального автомата управления при выполнении ветвления программы на основе анализа признаков состояния АЛУ.

На подготовительном этапе на вход регистра флагов поступает логическая единица (`flags_in = 1`), сигнализирующая о нулевом результате предыдущей арифметико-логической операции. На отметке 930 нс этот признак фиксируется в регистре, и на выходной линии `Z` устанавливается стабильный высокий уровень 1. Наличие данного флага является ключевым условием для последующего выполнения безусловного скачка по коду.

На фазе выборки команды (`DCa_out = 0001`) блок управления активирует сигналы `rom_en = 1` и `ir_load = 1`. Со сквозной шины памяти `rom_ram_out` считывается код команды 23, транслируемый из ПЗУ по адресу текущего счетчика `ip_out = 1`. По срезу импульса `ir_load` код операции защелкивается в регистре команд (`opcode` принимает значение 23), а адресное поле инструкции дешифрируется на шину `rom_addr = 2`. Блок управления распознает старшую тетраду кода как операцию условного перехода.

На фазе выполнения перехода (`DCa_out = 0002`) комбинационная логика автомата управления анализирует активное состояние флага нуля (`flags_out = 1`). Поскольку условие перехода истинно, блок управления одновременно генерирует импульсы `ip_load = 1` и `ip_en = 1`. Внутренний мультиплексор счетчика команд переключает цепь на сумматор смещения, который складывает текущий адрес со значением относительного сдвига из адресного поля команды, в результате чего значение счетчика `ip_out` скачкообразно изменяется с 1 сразу на 4. Системная шина `rom_ram_address` мгновенно дублирует этот адрес для подготовки выборки следующей инструкции из ПЗУ.

На последующих фазах цикла (`DCa_out = 0004` и `0008`) все исполнительные сигналы чтения/записи оперативной памяти и регистрового файла (`ram_en`, `dr_load`, `rom_we`) удерживаются автоматом управления в пассивном нулевом состоянии. Это исключает любые конфликты на общих шинах, пока процессор завершает текущий цикл распределителя тактов. На системной шине данных `rom_ram_out` в этот момент восстанавливается значение константы 240.

Полученные результаты сквозного моделирования полностью доказывают корректность совместной работы узла флагов и счетчика команд, подтверждают правильность логики управляющего автомата при выполнении как линейных операций считывания данных из памяти в РОН (`M->R`), так и логического ветвления по команде условного перехода (`JZ adr`).